

implement query processing and its execution. compare different indexing strategies. analyze transaction management in stored procedures.	Unit III: Advanced Relational Databases 3.1 Query Processing and Optimization 3.2 Advanced Indexing (B trees, Hash Indexing, Bitmap Indexing) 3.3 Materialized Views and Caching Strategies 3.4 Stored Procedures, Triggers, and UDFs	5 hrs
---	--	--------------

Er. Yuba Raj Devkota
NCIT

Query Processing and Optimization

Query processing involves converting a high-level SQL query into an efficient execution plan. Optimization aims to reduce query execution time and resource usage.

Stages of Query Processing

```
SELECT name FROM student WHERE age > 20;
```

1. Parsing & Translation

- SQL query → Relational Algebra expression.

- Example:

→ Relational Algebra:

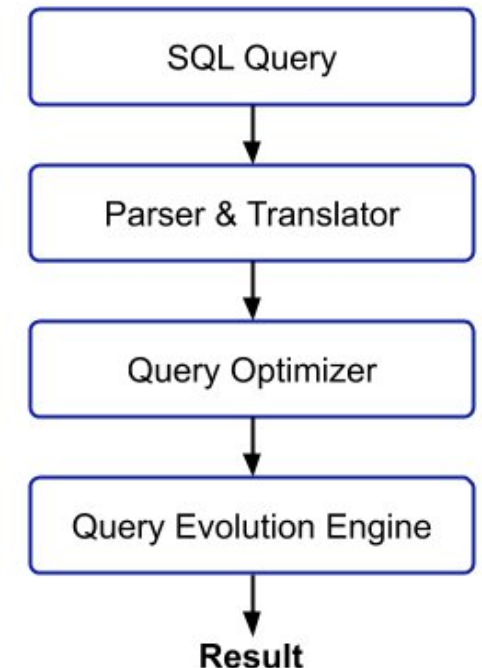
$$\pi_{name}(\sigma_{age>20}(student))$$

2. Query Optimization

- Logical Optimization (rewriting queries for efficiency).
- Physical Optimization (selecting best execution plan).

- Example:

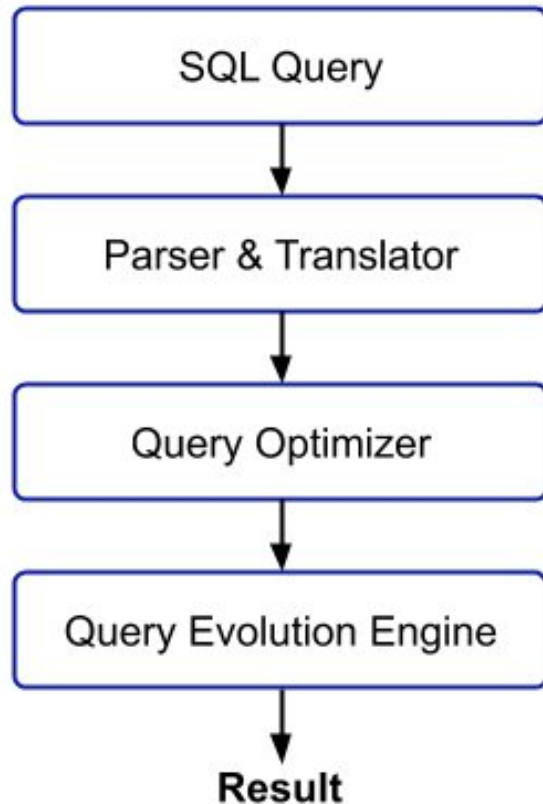
- Original: `SELECT * FROM student WHERE age > 20 AND dept = 'CS';`
- Optimized: Apply `dept = 'CS'` first if it filters more rows.



3. Execution Plan Generation

- Uses indexes, joins (nested-loop, hash, merge), and access methods.
- Example:

```
SELECT s.name, e.course  
FROM student s JOIN enrollment e ON s.id = e.student_id;
```



→ Execution Plan:

- Use **Hash Join** if tables are large.
- Use **Nested Loop Join** if one table is small.

Query Optimization Techniques

- **Cost-Based Optimization:** Estimates I/O, CPU, and memory usage.
- **Heuristic Optimization:** Applies rules (e.g., "perform selections early").
- **Join Ordering:** Minimizes intermediate result sizes.

Advanced Indexing

1. B-Tree Index Example

Scenario:

A table `students` with a high-cardinality column `student_id` (unique values).

How B-Tree Works:

- Stores keys in a **sorted, balanced tree structure**.
- Each node contains keys and pointers to child nodes.
- Supports **range queries** (e.g., `student_id BETWEEN 100 AND 200`).

```
SELECT * FROM students WHERE student_id = 150;
```

- B-tree traverses the tree to locate `150` in $O(\log n)$ time.

Why Use B-Tree?

- Ideal for **range queries** and **sorted data**.
- Default index in most databases (e.g., MySQL, PostgreSQL).

2. Hash Index Example

Scenario:

A table `users` with a column `email` (exact-match queries).

How Hash Index Works:

- Uses a **hash function** to map keys to fixed-size buckets.
- Directly retrieves data in **O(1)** time for exact matches.

Why Use Hash?

- Blazing fast for exact matches (e.g., =, IN).
- No support for range queries or sorting.

```
SELECT * FROM users WHERE email = 'alice@example.com';
```

- Hash function computes a bucket address for `alice@example.com`.

3. Bitmap Indexing

```
CREATE BITMAP INDEX idx_gender ON student(gender);
```

- Uses bitmaps for low-cardinality columns (e.g., gender, status).
- **Example:**
 - Helps in:

A column has **low cardinality** when it contains very few unique values compared to the total number of rows. Such as, male/female/others in gender. True/False in Boolean, Pending/completed/Active for status etc

```
SELECT * FROM student WHERE gender = 'F' AND status = 'active';
```

- **Advantage:** Efficient for **AND / OR** conditions.

Feature	B-Tree Index	Hash Index	Bitmap Index
Query Support	Range, equality	Equality only	Low-cardinality AND / OR
Speed	O(log n)	O(1)	O(1) for bitwise ops
Use Case	<code>student_id</code> , <code>date</code>	<code>email</code> , <code>primary key</code>	<code>gender</code> , <code>status</code>

Example of Bitmap Indexing

Scenario:

A table `employees` has a column `department` with **low cardinality** (only 3 unique values: "HR", "IT", "Finance").

Bitmap Index for `department` :

Each unique value gets a **bitmap** (a sequence of bits, where **1** = row has the value, **0** = does not).

Value	Bitmap (for rows 1–5)
HR	1 0 0 0 1
IT	0 1 0 1 0
Finance	0 0 1 0 0

Query Example:

Find all employees in **HR or IT**:

- HR bitmap: 1 0 0 0 1
- IT bitmap: 0 1 0 1 0
- OR operation: 1 1 0 1 1 → Matches rows 1, 2, 4, 5.

Table Data:

employee_id	name	department
1	Alice	HR
2	Bob	IT
3	Charlie	Finance
4	David	IT
5	Eve	HR

Why Use Bitmap?

- Efficient for AND/OR conditions (bitwise operations are fast).
- Saves space for low-cardinality columns (compared to B-trees).

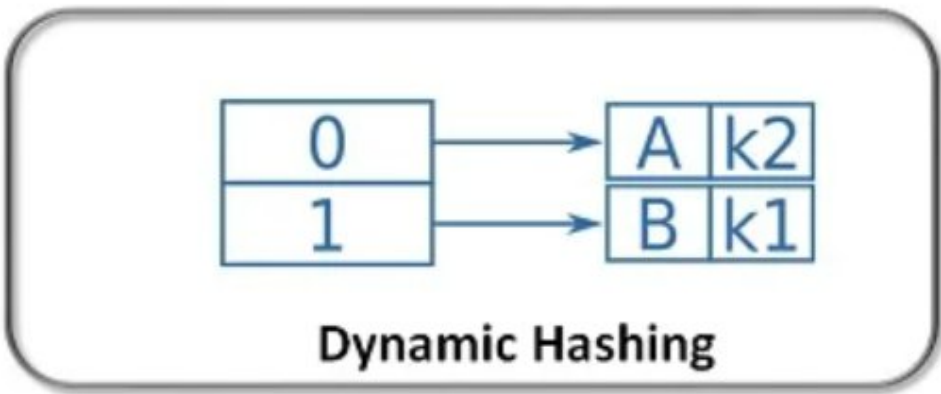
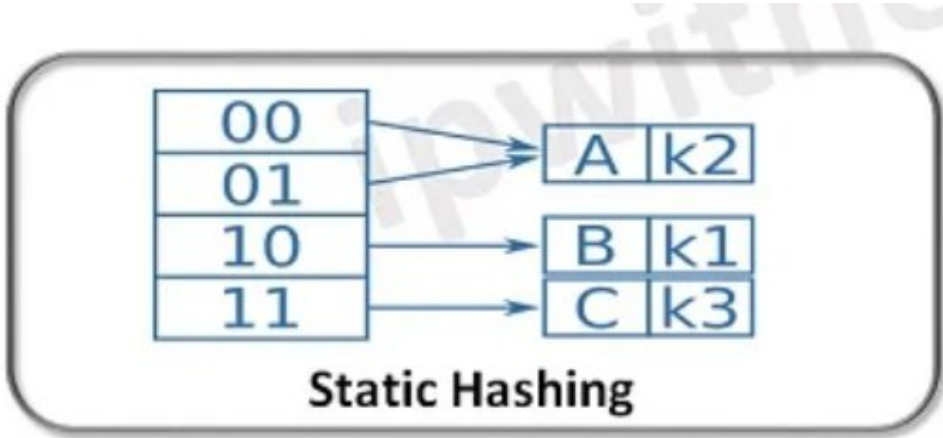
Limitation: Not suitable for high-cardinality columns (e.g., `employee_id`).

Syntax.

a) What are the limitations of static hashing in database systems? How does dynamic hashing address these limitations? Explain with suitable illustrations. 8

At the highest level, hashing techniques are classified based on whether the hash table's size can change at runtime.

- **Static Hashing:** The size of the hash table is fixed at the time of its creation and cannot be changed.
- **Dynamic Hashing:** The size of the hash table can be dynamically resized as needed to accommodate more data.



Limitations of Static Hashing

1. Fixed number of buckets

- In static hashing, the number of buckets is fixed at the time of file creation.
- If data grows, many records go to the same bucket → **overflow chains**, causing slower search.

2. Wasted space when data decreases

- If records are deleted, many buckets remain empty → **poor space utilization**.

3. Performance degrades over time

- As overflow increases, search/insert time becomes slow.

How Dynamic Hashing Solves It

Dynamic Hashing (like **Extendible Hashing** or **Linear Hashing**) grows and shrinks the number of buckets **automatically** based on the amount of data.

Improvements:

- **Expands** when buckets overflow → fewer overflow chains.
- **Shrinks** when data decreases → saves space.
- Maintains **fast search time ($O(1)$)** even when data grows.

Static Hashing Example

Hash function:

$$h(\text{key}) = \text{key} \bmod 4$$

Buckets = B0, B1, B2, B3 (fixed)

Insert keys: 10, 14, 18, 22

All give:

- $10 \bmod 4 = 2$
- $14 \bmod 4 = 2$
- $18 \bmod 4 = 2$
- $22 \bmod 4 = 2$

*All go to Bucket B2 →
overflow chain starts →
search becomes slow.*

Key Idea: Dynamic Hashing DOES NOT use only the last bit... it uses MORE bits when needed

At the beginning, it uses **1 bit**.

If overflow happens, it expands the directory and uses **2 bits**.

If still overflow, it uses **3 bits**, and so on.

So when buckets overflow, **we look at more bits**, not only the last bit.

Using last 1 bit:

- 10 → 0 → A
- 14 → 0 → A
- 18 → 0 → A
- 22 → 0 → A

👉 Bucket A becomes full (overflow).

Now use last 2 bits:

Key	Binary	Last 2 bits	Goes to
10	1010	10	B
14	1110	10	B
18	10010	10	B
22	10110	10	B

Yes — all STILL go to one bucket (Bucket B).

So Bucket B now overflows.

Now use last 3 bits:

Key	Binary	Last 3 bits	Bucket
10	1010 → 010	Bucket 010	
14	1110 → 110	Bucket 110	
18	10010 → 010	Bucket 010	
22	10110 → 110	Bucket 110	

Now the keys SPLIT!!

- 10 and 18 go to Bucket **010**
- 14 and 22 go to Bucket **110**

👉 No overflow!

- b) How does a materialized view differ from a standard (regular) view in a database? Explain materialized views and caching strategies in detail.

Materialized Views

- *A Materialized View is a physical copy of a query result stored in the database.*
- *It contains actual data, not just the SQL query.*
- *It is refreshed periodically (manually or automatically).*

Feature	Regular View	Materialized View
Storage	Does NOT store data (virtual)	Stores data physically
Speed	Slow (query runs every time)	Fast (precomputed results)
Data Freshness	Always up-to-date	May be old (stale) until refreshed
Refresh	No refresh needed	Needs refresh (ON DEMAND or AUTOMATIC)
Usage	Useful for dynamic data	Useful for costly/large queries

Suppose you have large tables:

- Sales (millions of rows)
- Customers

You want monthly total sales.

```
CREATE VIEW monthly_sales AS
SELECT customer_id, SUM(amount) AS total
FROM sales
GROUP BY customer_id;
```

Regular View: ➔ *Every time you query the view, the entire SUM must be recalculated → slow.*

```
CREATE MATERIALIZED VIEW monthly_sales_mv AS
SELECT customer_id, SUM(amount) AS total
FROM sales
GROUP BY customer_id;
```

Materialized View:

- ➔ Data is stored physically.
- ➔ Query becomes very fast because sums are already calculated.

Caching Strategies for Materialized View

Caching means: How often and when should the materialized view update its data?

1. Full Refresh

Recalculates the entire view.

- Slow but accurate.
- Used when data changes a lot.

Example:

```
REFRESH MATERIALIZED VIEW monthly_sales_mv;
```

2. Fast Refresh

Updates only changed (delta) rows.

- Very fast.
- Needs logs or triggers.

Example:

```
CREATE MATERIALIZED VIEW monthly_sales_mv  
REFRESH FAST ON COMMIT;
```

➡ Updates automatically whenever base table changes.

3. Refresh on Commit (Real-time)

Every insert/update on base table triggers refresh.

- Most fresh
- Slight overhead on transactions

sql

```
REFRESH FAST ON COMMIT;
```

4. Scheduled Refresh

Refresh at fixed intervals.

- Good balance between freshness and performance.

sql

```
REFRESH FAST
```

```
START WITH SYSDATE
```

```
NEXT SYSDATE + 1; -- refresh daily
```

Example of Caching Strategy

You run a website where:

- Sales table updates every minute.
- Managers see sales dashboard every hour.

Materialized View Strategy:

- Use **FAST REFRESH** every 15 minutes.

This gives:

- Faster dashboard loading
- Data only 0–15 minutes old
- No heavy recalculation each time

Stored Procedures

- Precompiled SQL code stored in the database for reuse.
- Improves performance (reduces network traffic) and security (avoids SQL injection).

We have a table `employees` and want to create a stored procedure to **get employees by department** and then **call it**.

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(50),  
    salary DECIMAL(10, 2)  
);  
  
INSERT INTO employees VALUES  
(1, 'Alice', 'HR', 50000),  
(2, 'Bob', 'IT', 60000),  
(3, 'Charlie', 'Finance', 70000),  
(4, 'David', 'IT', 65000),  
(5, 'Eve', 'HR', 55000);
```

```
-- Procedure to fetch employees by department  
DELIMITER //  
CREATE PROCEDURE GetEmployeesByDept(IN dept_name VARCHAR(50))  
BEGIN  
    SELECT * FROM employees WHERE department = dept_name;  
END //  
DELIMITER ;
```

- `DELIMITER //`: Temporarily changes the delimiter (since `;` is used inside the procedure).
- `IN dept_name`: Input parameter (the department name to filter).
- `BEGIN ... END`: Contains the SQL logic.

```
-- Get all IT employees
CALL GetEmployeesByDept('IT');
```

Expected Output:

emp_id	emp_name	department	salary
2	Bob	IT	60000
4	David	IT	65000

Modify Procedure (Bonus - Add Salary Filter)

```
DELIMITER //
CREATE PROCEDURE GetEmployeesByDeptAndSalary(
    IN dept_name VARCHAR(50),
    IN min_salary DECIMAL(10, 2)
)
BEGIN
    SELECT * FROM employees
    WHERE department = dept_name AND salary >= min_salary;
END //
DELIMITER ;

-- Call: Get IT employees earning ≥ 62000
CALL GetEmployeesByDeptAndSalary('IT', 62000);
```

Output:

emp_id	emp_name	department	salary
4	David	IT	65000

Triggers

- Automatically executes SQL code when a specified event (INSERT, UPDATE, DELETE) occurs.
- Used for **audit logs**, **data validation**, and **auto-updates**.

Let's create an audit log that automatically records changes made to an `employees` table.

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(50),  
    salary DECIMAL(10, 2)  
);
```

```
INSERT INTO employees VALUES  
(1, 'Alice', 'HR', 50000),  
(2, 'Bob', 'IT', 60000);
```

Create an Audit Log Table

```
CREATE TABLE employee_audit_log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT,  
    changed_column VARCHAR(50),  
    old_value VARCHAR(100),  
    new_value VARCHAR(100),  
    change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Create a Trigger

This trigger will log salary changes:

```
DELIMITER //
CREATE TRIGGER log_salary_changes
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary != NEW.salary THEN
        INSERT INTO employee_audit_log (emp_id, changed_column, old_value, new_value)
        VALUES (NEW.emp_id, 'salary', OLD.salary, NEW.salary);
    END IF;
END //
DELIMITER ;
```

Test the Trigger

```
-- Update an employee's salary
UPDATE employees SET salary = 55000 WHERE emp_id = 1;

-- View the audit log
SELECT * FROM employee_audit_log;
```

Expected Output in Audit Log

log_id	emp_id	changed_column	old_value	new_value	change_time
1	1	salary	50000	55000	2023-10-25 14:30

User-Defined Functions (UDFs)

Scenario: Let's create a function that calculates an employee's bonus (10% of salary) and use it in a query.

- Custom functions that return a value (scalar or table).
- Can be used in SQL queries.

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary DECIMAL(10, 2)  
);
```

```
INSERT INTO employees VALUES  
(1, 'Alice', 50000),  
(2, 'Bob', 60000),  
(3, 'Charlie', 75000);
```

Create a UDF to Calculate Bonus

```
DELIMITER //  
CREATE FUNCTION CalculateBonus(emp_salary DECIMAL(10,2))  
RETURNS DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    DECLARE bonus DECIMAL(10,2);  
    SET bonus = emp_salary * 0.10; -- 10% bonus  
    RETURN bonus;  
END //  
DELIMITER ;
```

Use the Function in a Query

```
SELECT
    emp_name,
    salary,
    CalculateBonus(salary) AS bonus
FROM employees;
```

Expected Output

emp_name	salary	bonus
Alice	50000	5000
Bob	60000	6000
Charlie	75000	7500

UDF vs. Stored Procedure

Feature	UDF	Stored Procedure
Return Value	Required	Optional
Usage	In SELECT/WHERE clauses	Called with CALL
Transactions	Not supported	Supported

Summary

- **Materialized Views** improve read performance for complex queries.
- **Caching** (Redis, DB cache) speeds up frequent queries.
- **Stored Procedures** encapsulate reusable SQL logic.
- **Triggers** automate actions on data changes.
- **UDFs** allow custom functions in SQL queries.

Homework [SOE, pre-board]

3. Define stored procedure. How can we create parameterized stored procedure? Explain with necessary syntax and example.
4. Define Trigger and its role in maintaining data integrity. In a conference Management System that manages authors, papers, reviewers, sessions and scheduling, how can trigger be utilized to enforce constraint “A reviewer can review a maximum of 5 papers”.
5. Write the algorithm for insertion in B+ Index Tree. Construct B+ tree of order 4 for the following set of keys, 2,3,5,7,11,17,19,23,29,31.
6. In a university database, how can you efficiently get the total salary paid to instructor in each department without recalculating the sum every time? Write necessary SQL queries to create, refresh and view for this purpose.

B+ Tree is a balanced multi-level index structure used in databases and file systems to store and search large amounts of sorted data efficiently.

B+ Tree is an extension of the B-Tree where:

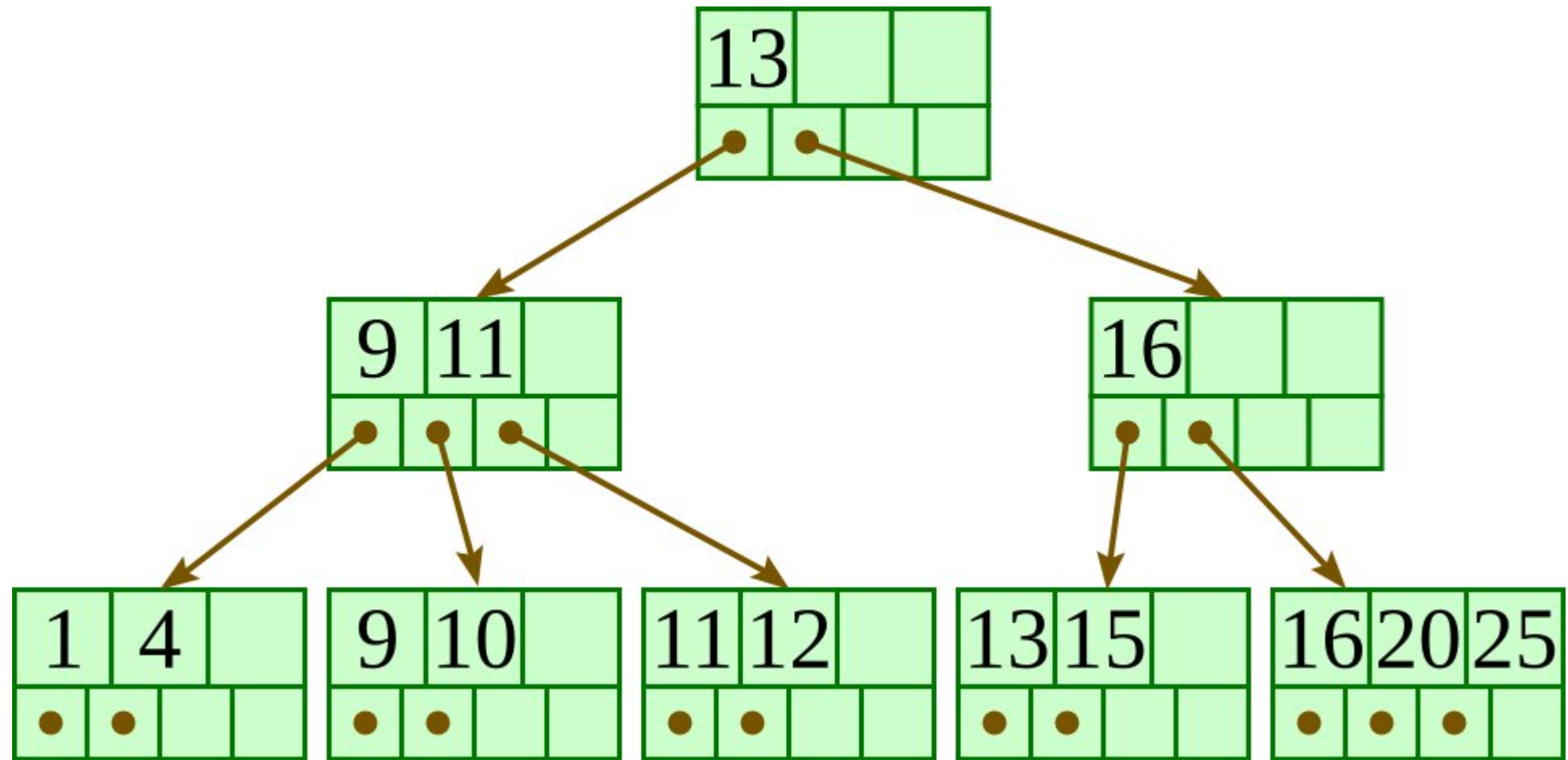
- All actual data records are stored only in the leaf nodes.
- Internal nodes (non-leaf nodes) contain only keys for indexing, not data.
- Leaf nodes are linked like a linked list, making range queries fast.
- The tree remains balanced, meaning all leaves appear at the same level.

✓ Algorithm for Insertion in a B+ Tree (Order = m)

(Here $m = 4 \rightarrow$ max 3 keys per node, max 4 children)

Steps

1. Search the leaf node where the new key should be inserted.
2. Insert the key in sorted order inside the leaf.
3. If the leaf is not full, DONE.
4. If the leaf overflows (i.e., contains $> m-1$ keys):
 - Split the leaf into two nodes:
 - Left node: first $\lceil m/2 \rceil$ keys
 - Right node: remaining keys
 - Copy the first key of the right node to the parent.
5. Insert the promoted key in the parent in sorted order.
6. If parent overflows, split it similarly.
7. If the root splits, create a new root with two children.
8. Continue until all splits are resolved.



✓ Construct B+ Tree of Order 4 (max 3 keys per node)

Keys: 2, 3, 5, 7, 11, 17, 19, 23, 29, 31

